

Improved-Engineering-OS

דוח ארכיטקטורה סופי ותוכנית מימוש מבוססת ראיות

Architecture Baseline · Version 1.0

Asia/Jerusalem · 2026 בספטמבר 4

17 החלטות ארכיטקטוניות מאושרות

Knowledge · Evidence · Assurance · Releases · Agent-Neutral Runtime

תוכן עניינים

- 1 . תקציר מנהלים ומעמד המסמך
- 2 . חזון, רקע ועקרונות התכנון
- 3 . ארכיטקטורת היעד וחוקת המערכת
- 4 . ההחלטות המאושרות — D1-D17
- 5 . תוכנית המימוש מבוססת הסימולציות — Stage 0-17
- 6 . כשירות, תפעול ושחרור
- 7 . נספחים וצעדים הבאים

הערה: מספרי עמודים עשויים להשתנות בין Word, LibreOffice ו־PDF; הניווט מבוסס על היררכיית הכותרות.

תקציר מנהלים ומעמד המסמך

שם הפרויקט: Improved-Engineering-OS

גרסת מסמך: 1.0 — Architecture Baseline

תאריך בסיס: 4 בספטמבר 2026, Asia/Jerusalem

סטטוס: 17 החלטות ארכיטקטוניות מאושרות; המסמך מאחד אותן לחוזה מערכת אחד.

מקור החלטות: תהליך התכנון שאושר בשיחה זו, כולל הבהרות לגבי שימוש בנכסי Engineering OS הקיימים, פתרון קנוני יחיד לבעיה שקולה, אוטונומיית agents, למידה מ־telemetry ו־AI-progress-native architecture.

מקור מחקר: תיעוד רשמי שנבדק מחדש לצורך מסמך זה: NIST SSDF, OpenTelemetry, Model Context, Anthropic, OpenAI, Supabase, Protocol, GitHub. מידע חיצוני שיכול להשתנות לאחר 4 בספטמבר 2026 כפוף למדיניות Live Verification של הפרויקט.

Provenance עתידי: לאחר הכנסת הדוח ל־Git יש להוסיף ל־metadata של המסמך git_commit, content_hash, architecture_version ו־sources_verified_at. אין להמציא SHA לפני שקיים commit אמיתי.

מטרת הפרויקט היא לבנות מערכת אישית, מתמשכת ומבוססת ראיות שמגדילה את איכות ומהירות עבודת הפיתוח עם agents כגון Claude ו־Codex, מבלי להפוך ל־workflow engine שמנהל אותם. המערכת מחזיקה את מה שכדאי שיהיה יציב מחוץ ל־agent — ידע, Project Context, Evidence, Assurance, היסטוריה, Releases ולקחים — ומשאירה ל־agent את מה ש־AI משתפר בו במהירות: תכנון, בחירת כלי, חלוקת עבודה, שימוש ביכולות native ואסטרטגיית ביצוע.

העיקרון העליון הוא:

כלל שה־AI מתקדם, Improved-Engineering-OS צריך להיות מסוגל לעשות פחות orchestration ולהפיק יותר ערך מהידע, הראיות והזיכרון המצטברים שלו.

לכן ארכיטקטורה מוצלחת אינה זו שמשמרת לנצח את כל ה־subsystems שנבנו ב־2026. הצלחה כוללת גם יכולת לזהות באמצעות evals שכבה שהפכה מיותרת, להסיר אותה, ולאפשר ל־agent חדש וחזק יותר לעבוד בצורה טבעית יותר.

המערכת מתחילה **מכל נכסי הידע הקיימים של Engineering OS הישן**, אך אינה יורשת את מימוש המערכת הישנה. העיקרון הוא:

את הידע אנחנו יורשים; את המערכת אנחנו בונים מחדש.

במקביל, Improved-Engineering-OS לא אמור לחשוף ל־agent חמישה פתרונות כמעט זהים. עבור אותה בעיה, אותם constraints ואותו compatibility context, ברירת המחדל היא **Champion אחד — הפתרון הקנוני הטוב ביותר שהראיות מצדיקות כרגע**. Challengers והיסטוריית פתרונות נשמרים מאחור כדי לאפשר החלפה עתידית, אבל אינם יוצרים עומס בשימוש היומיומי.

הארכיטקטורה נשענת על חמישה boundaries עיקריים:

Canonical Git

= מה המערכת יודעת, מגדירה ומשחררת באופן קנוני

Target Project Git

= מה הפרויקט הספציפי מתכוון להיות והגרסה שאליה הוא מוצמד

Local Runtime

= ובמחשב המקומי agent-מה קורה עכשיו אצל ה

Supabase Evidence Plane

= מה באמת קרה ומה למדנו ממנו

External World

= freshness/risk מה עשוי להיות נכון עכשיו ודורש אימות לפי

אין מסלול ישיר:

AI / Internet / MCP / Agent Observation

↓
Canonical Truth

המסלול הקנוני הוא:

Observation
↓
Curation
↓
Candidate
↓
Validation + Evidence
↓
Promotion
↓
Canonical Git
↓
Future pinned release

זהו boundary יסודי נגד knowledge poisoning, drift ו־self-modifying governance.

מבחינת סטנדרטים, הבחירה לשמור את ה־domain model של EOS יציב ואת transports מתחלפים מתחזקת במיוחד מכך ש־MCP עצמו עבר ביולי 2026 שינוי משמעותי לליבה stateless, הוסיף caching ו־extensions ושינה/deprecated יכולות קודמות; כלומר אפילו פרוטוקול מרכזי ועדכני הוא שכבת integration שמתפתחת ולא domain abstraction שכדאי לקשור אליו את ליבת הפרויקט. OpenTelemetry מספקת מודל מתאים לחוזים ארוכי־חיים: Development → Stable → Deprecated → Removed, עם backward-compatibility לחוזים Stable ו־major version עבור שבירה.

הדוח מגדיר ארכיטקטורה ארוכת טווח, אך **בכוונה אינו מקבע מספרים שאין עבורם עדיין data אמיתי**, כגון retention של RPO/RTO, margin, raw telemetry להחלפת Champion, משקלי Asset Score, מספר trials קבוע או תקציב tokens מוחלט. אלה יהפכו ל־versioned policies לאחר ש־Stages המוקדמים ייצרו baselines.

חזון, רקע ועקרונות התכנון

ה־Engineering OS הקודם צבר נכסים ולקחים שימושיים, אך התפתח סביב הנחות שהתאימו יותר לדור קודם של agents. הרטרוספקטיבה של הפרויקט זיהתה כמה דפוסים שלא רצוי לשמר כמבנה מערכת:

יותר מדי governance סביב Routing. execution מחייב, planning phases, הפעלה חובה של Skills, Route Plans, matrices של instructions ו־workflow policing ניסו לפצות על limitations של agents במקום להניח שיכולותיהם ישתפרו.

עומס וכפילויות. כאשר כמה Patterns, Skills או reference solutions עוסקים למעשה באותה בעיה, עצם הצגתם כאפשרויות שוות מעבירה את עבודת הסינון חזרה ל־agent ומגדילה context ללא ערך.

הוכחה חלקית מדי. Unit tests ירוקים יכולים להוכיח function מסוים אך לא wiring, התקנה נקייה, התאוששות, שימוש טבעי ב־agent או ערך אמיתי בפרויקט. Improved-Engineering-OS דורש simulation ו־real-project qualification לפני promotion.

תלות יתר בממשקי agent/tool של תקופה מסוימת. MCP, plugins, Skills, hooks, CLAUDE.md, AGENTS.md או כלי native הם delivery mechanisms. אף אחד מהם אינו ה־domain model של EOS.

למידה ללא גבול ברור בין Observation לאמת. מערכת שמשנה canonical knowledge ישירות אחרי Run מוצלח אחד תיצור מהר poisoning, feedback loops ו־false confidence.

ערבוב של איכות, התאמה והרשאה. נכס איכותי אינו בהכרח מתאים לפרויקט; integration זמין אינו בהכרח מורשה; provider טוב אינו בהכרח הבחירה הארכיטקטונית הרצויה; ו־registry רשמי אינו security certification. ה־MCP Registry הרשמי עצמו מציג metadata ו־namespace authenticity, אך מגדיר את metadata כ־unopinionated ומעביר security scanning ל־package registries ו־downstream aggregators.

מכאן נגזרים עקרונות היסוד:

.EOS subsystem יכולה להחליף של **AI-progress-native** — Native capability
 .execution מנהל agent; ה־evidence constraints; ידע, **Knowledge over workflow** — EOS
Capability ≠ Provider ≠ Integration — צורך, ספק, דרך חיבור וחיבור runtime הם ישויות שונות.
One equivalent problem → one visible Champion — אין clutter של פתרונות זהים; challengers נשמרים להערכה.
Evidence before authority — Score גבוה לבדו לא הופך asset לקנוני או מאפשר bypass של safety.
Progressive disclosure — resolve מחזיר מעט; inspect פותח עומק; expand מרחיב רק בעת צורך.
Fail closed only where consequence demands it — coding שגרתי לא נחסם בגלל telemetry backend; gate קריטי אינו fail-open.
No silent guessing — חוסר מידע הוא UNKNOWN, לא inferred success.
One state, one owner — אין שני SSOTs לאותו fact.
Pinned baseline + live overlay — reproducibility freshness מתקיימים בו־זמנית.
Everything consequential is explainable — Scores, Controls, Recommendations promotions ניתנים לתחקור.
Build small, prove, then expand — כל stage מוכיח claim לפני שהמערכת מסתמכת עליו.
 הגישה של NIST SSDF מחזקת מודל שבו security practices משתלבים בכל SDLC מבלי להכתיב מודל lifecycle יחיד, ומתמקדת בתוצאות כגון הפחתת vulnerabilities ומניעת recurrence — התאמה טובה ל־Assurance ול-simulation gates שלנו במקום workflow אבטחה קשיח.
Non-goals מפורשים של Improved-Engineering-OS v1: אינו enterprise platform, אינו multi-tenant, אינו governance system, אינו marketplace של MCPs, אינו universal proxy ל־GitHub/Supabase/Vercel, אינו provider abstraction שמסתיר יכולות native, אינו orchestration framework שמחלק כל task ל־steps, ואינו catalog של "כל מה שקיים באינטרנט". הוא מערכת אישית עם abstractions שניתן להצדיק באמצעות שימוש וראיות.

השוואת שכבות Capability:

Capability

השאלה: מה צריך לעשות?

דוגמה: database.schema.read

יציבות צפויה: גבוהה

Provider / Offering

השאלה: מי מספק את השירות?

דוגמה: Supabase Postgres

יציבות צפויה: בינונית

Integration

השאלה: איך מתקשרים עם השירות?

דוגמה: native connector / MCP / CLI

יציבות צפויה: נמוכה-בינונית

Runtime Connection

השאלה: מה זמין, healthy ומורשה עכשיו?

דוגמה: Supabase connector, read-only, authenticated

יציבות צפויה: אפמרלית

השוואת agents:

EOS מול Contract

Claude: resolve / inspect / expand

Codex: זהה

Agent עתידי: זהה ככל האפשר

Runtime capabilities

Claude: מתגלות בתחילת Run

Codex: מתגלות בתחילת Run

Agent עתידי: negotiation, לא hard-code

Plugins/MCP/native tools

Claude: לפי מה שקיים בפועל

Codex: לפי מה שקיים בפועל

Agent עתידי: יכול לכלול modality שלא קיימת כיום

בחירת execution tool

Claude: agent

Codex: agent

Agent עתידי: agent

State קנוני

Claude: מחוץ ל-agent

Codex: מחוץ ל-agent

Agent עתידי: מחוץ ל-agent

Adapter

Claude: דק

Codex: דק

Agent עתידי: חדש רק עבור delivery/negotiation

EOS-specific workflow

Claude: אין

Codex: אין

Agent עתידי: אין

הגישה תואמת למגמה שכבר קיימת בפלטפורמות agents: למשל Anthropic תומכת ב־deferred tool loading כך שמספר רב של tools יכול להישאר מחוץ ל־context ולהיטען כאשר הם מתגלים כרלוונטיים, במקום להכניס מראש את כל הגדרות הכלים. במקביל Anthropic מזהירה במפורש ש־remote MCP servers הם שירותי צד שלישי ושיש להעריך את practices והאמון בהם לפני חיבור.

השוואת integrations:

Native agent capability

תפקיד: שימוש ביכולת מובנית

יתרון: לעיתים UX/context הטובים ביותר

סיכון/מגבלה: משתנה במהירות עם model/platform

סטטוס ארכיטקטוני: Preferred כאשר מוכח

Native provider connector/plugin

תפקיד: גישה מובנית לספק

יתרון: structured access

סיכון/מגבלה: platform-specific

סטטוס ארכיטקטוני: Runtime option

MCP

תפקיד: פרוטוקול סטנדרטי לכלים/resources

יתרון: portability/discovery

סיכון/מגבלה: specification מתפתחת, trust נפרד

סטטוס ארכיטקטוני: Adapter/transport

CLI

תפקיד: fallback ואוטומציה
 יתרון: פשוט, debug-friendly
 סיכון/מגבלה: parsing/auth/environment
 סטטוס ארכיטקטוני: Fallback/runtime option

Direct API/SDK

תפקיד: שליטה מדויקת
 יתרון: capability מלאה
 סיכון/מגבלה: maintenance/auth
 סטטוס ארכיטקטוני: Task-dependent

Browser/web

תפקיד: discovery/live verification
 יתרון: freshness
 סיכון/מגבלה: untrusted content, פחות structured
 סטטוס ארכיטקטוני: Verification/discovery

Skill

תפקיד: guidance/procedure reusable
 יתרון: יכול להפחית retries
 סיכון/מגבלה: יכול להפוך overhead אם AI משתפר
 סטטוס ארכיטקטוני: Asset, לא חובה

ארכיטקטורת היעד וחוקת המערכת

ה־Architecture Constitution אינה Decision נוספת. היא החוזה שחוצה את כל 7 1 ההחלטות ומונע מהן להתפרש בדרכים סותרות.

```

flowchart TB
    EXT["External World<br/>Docs · Providers · MCP · APIs · Repos"]
    LIVE["Live Verified Overlay"]
    RUNTIME["Local Runtime Plane<br/>Launcher · Resolver · Adapters<br/>Capability Discovery · SQLite Outbox"]
    AGENTS["Claude · Codex · Future Agents"]
    SUPA["Supabase Evidence Plane<br/>Telemetry · Observations · Evidence<br/>Evals · Investigations · Candidates"]
    PROMO["Curation & Promotion Pipeline"]
    CORE["Canonical Git Plane<br/>Code · Schemas · Knowledge · Controls<br/>Skills · Evals · Migrations · Releases"]
    PROJECT["Target Project Git<br/>EOS Pin · Project Profile spec · ADRs"]

    EXT --- LIVE
    LIVE --- RUNTIME
    RUNTIME --- AGENTS
    AGENTS --- SUPA
    SUPA --- PROMO
    PROMO --- CORE
    CORE --- PROJECT
  
```

```

EXT -->|verify volatile facts| LIVE
LIVE --> RUNTIME
CORE -->|pinned immutable release| RUNTIME
PROJECT --> RUNTIME
RUNTIME <--> AGENTS
RUNTIME -->|durable telemetry| SUPA
EXT -->|observations only| SUPA
SUPA --> PROM0
PROM0 -->|validated canonical change| CORE
CORE -->|new release| PROJECT

```

State Ownership Matrix

Core code, schemas, migrations

SSOT / Owner: Canonical Git

Mutable by: controlled code change

Versioned: הערה:

Canonical Knowledge Assets

SSOT / Owner: Canonical Git

Mutable by: Promotion Path

הערה: Agent לא כותב ישירות

Controls/Assurance definitions

SSOT / Owner: Canonical Git

Mutable by: controlled promotion

הערה: שינוי authority נחשב consequential

Eval/Simulation definitions

SSOT / Owner: Canonical Git

Mutable by: controlled code/promotion

הערה: hidden answers מחוץ ל-agent workspace

Release manifests

SSOT / Owner: Canonical Git/immutable release artifact

Mutable by: release process

הערה: Pin לפי digest/version

Project Profile spec

SSOT / Owner: Target Project Git

Mutable by: user/approved project change

הערה: desired/declared state

ADRs/Open Decisions resolved

SSOT / Owner: Target Project Git

Mutable by: user/project workflow

הערה: durable intent

Observed project state

SSOT / Owner: Runtime/Evidence Plane

Mutable by: probes/runtime

הערה: לא משוכתב כל הזמן ל-Git

Runtime capability snapshot

SSOT / Owner: Current Run

Mutable by: adapter/agent runtime

הערה: ephemeral

Pending telemetry

SSOT / Owner: Local SQLite

Mutable by: EOS runtime

הערה: durable until acknowledged

Raw retained telemetry

SSOT / Owner: Supabase Evidence Plane

Mutable by: ingestion path

הערה: append-oriented

Derived Evidence

SSOT / Owner: Supabase Evidence Plane

Mutable by: versioned derivers

הערה: replayable

Asset Score

SSOT / Owner: Derived view/state

Mutable by: scoring algorithm

הערה: לא ידנית

Observations/Candidates

SSOT / Owner: Evidence Plane

Mutable by: runs/curator

הערה: אינם canonical

Current external facts

SSOT / Owner: external source + live overlay

Mutable by: verifier

הערה: לא SSOT קנוני של EOS

Local cache

SSOT / Owner: Local Runtime

Mutable by: runtime

הערה: disposable

Project intent לעומת Project Profile **reality**. צריך להפריד conceptually בין spec ובין observed status. Git שומר את המצב המוצהר; probes ו־Evidence Plane שומרים מה נצפה. אין כלל של "user תמיד מנצח" או "repo תמיד מנצח" במקרה של סתירה:

Declared == Observed
→ ALIGNED

Declared != Observed
→ DRIFT / CONFLICT

Not enough evidence
→ UNKNOWN

לדוגמה, אם database_provider: supabase הוצהר אך Firebase config פעיל עדיין, זו אינה סיבה לנחש מי "נכון"; זו Architecture Drift שצריך להציג.

.Live Overlay מול Pinned baseline

```

flowchart LR
    PIN["Pinned EOS Release<br/>reproducible baseline"]
    LIVE["Live Verified Fact<br/>source + retrieved_at + freshness"]
    TASK["Current Agent Decision"]
    OBS["Observation"]
    CUR["Curation"]
    NEXT["Future Canonical Release"]

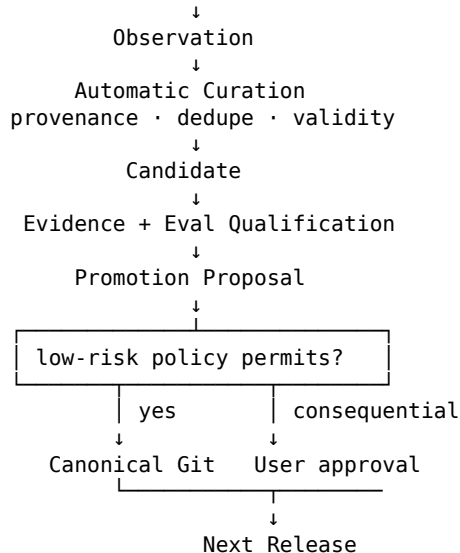
    PIN --> TASK
    LIVE --> TASK
    LIVE --> OBS --> CUR --> NEXT
  
```

ה־release המוצמד לפרויקט מכיל את ה־logic ו־knowledge baseline, Controls, schemas שניתנים לשחזור. מידע כגון מחיר נוכחי, API limit, service status, supported feature או deprecation עשוי לדרוש live verification. אותו מידע יכול להשפיע על ה־Run הנוכחי אך **אינו משנה בשקט את ה־release המוצמד**. שינוי קנוני עובר $\text{Observation} \rightarrow \text{Curation} \rightarrow \text{Release}$ עתידי.

זו מדיניות חשובה במיוחד לאינטגרציות: ה־MCP Registry הרשמי עדיין נמצא ב־preview נכון לתאריך הדוח ומזהיר במפורש ש־breaking changes או resets אפשריים לפני GA, ולכן registry metadata אינו בסיס טוב ל־canonical runtime state.

Canonical Write Path

Run / CI / External Change / Human Correction



אין agent learned $\rightarrow$ edit canonical knowledge.

קידום אוטומטי עתידי מותר רק ללקטגוריות low-risk שנקבעו במפורש. שינוי ב־GATE/DENY, executable Skill, provider architectural או schema semantics, scoring semantics, permission model, security authority default דורש אישור אנושי.

Trust Vector. אין trusted=true.

publisher_authenticity — האם המקור באמת שייך למי שהוא טוען

provenance_integrity — האם ניתן לקשור את התוכן/version למקור יציב

content_quality — האם התוכן מדויק ושימושי

security_status — vulnerability/risk state ידוע

evidence_strength — הצלחה אמפירית ב־EOS

freshness — מתי fact נבדק ומה volatility שלו

runtime_permission — מה מותר לחיבור לבצע כרגע

blast_radius — READ_ONLY / WRITE / DESTRUCTIVE / PRIVILEGED

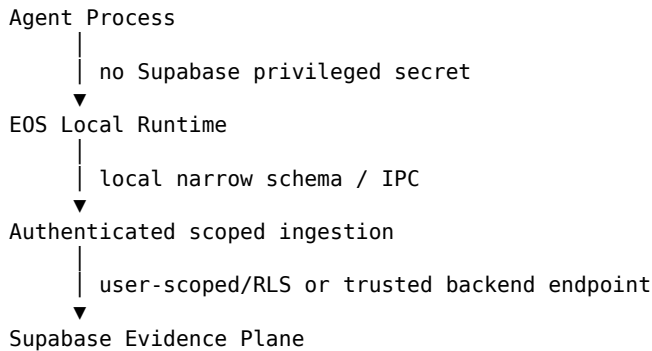
Authenticity אינה Security. Security אינה Quality. Quality אינה Permission.

ההבחנה חשובה במיוחד ב־MCP: ה־Official Registry מאמת namespaces ומאפשר discovery, אך security scanning של הקוד עצמו נעשית מחוץ ל־registry.

גבול הרשאות Supabase. אסור שה־agent יקבל secret/service-role credential של Evidence Plane. נכון ל־4 בספטמבר 2026, Supabase עוברת מ־legacy anon/service_role ל־publishable/secret keys עד

סוף 2 0 2 6 ; secret keys ממופים להרשאת service_role, עוקפים RLS, ו-Supabase מזהירה שדליפת secret key חושפת את כלל נתוני הפרויקט.

הארכיטקטורה המועדפת היא:



ה-agent אינו מקבל database-admin credential דרך prompt, .env, Project Profile, Knowledge Asset או authority. Ingestion, read/analytics, telemetry, curator/admin ו-identities נפרדים מבחינת Secret key, אם נדרש בצד server/Edge Function, נשאר ברכיב trusted backend ולא בסביבת agent. Publishable credentials יחד עם Auth/RLS או endpoint ייעודי מועדפים עבור local ingestion כאשר הם מאפשרים את ה-scope הנדרש. ההכרעה הטכנולוגית המדויקת תיעשה ב-5 Stage תוך שמירת boundary זה.

:Agent permission invariant

AVAILABLE
 ≠
 RECOMMENDED
 ≠
 AUTHORIZED

resolve יכול להמליץ על integration; recommendation לעולם אינה מעניקה permission.

Data Lifecycle

Evidence-critical compact records
 → durable / replayable

Raw diagnostic telemetry
 → Hot → Warm/Cold → expiry policy

Local outbox
 → delete only after durable acknowledgement

Caches
 → disposable

אין לקבע retention numbers לפני volume אמיתי, אך retention policy חייבת להיות versioned. הנדרש לשחזור Attribution או Asset Score צריך להישאר מספיק זמן כדי לאפשר replay גם אחרי מחיקת diagnostic noise.

Backup חייב לכסות את כל Supabase storage classes מבהירה ש-Database Backups אינם כוללים את האובייקטים עצמם שנשמרו דרך Storage API; ה-database מחזיק רק metadata. לכן אם cold telemetry Postgres יעבור ל-Supabase Storage, backup/restore שלו חייב להיות נפרד מגיבוי Postgres.

הכלל:

Backup שלא עבר restore simulation אינו Evidence ל-recoverability.

RPO/RTO ייקבעו לפני שה-Evidence Plane נהיה תלות production משמעותית.

Telemetry Causality. Investigation לא יכול להסתמך רק על wall-clock timestamp. אירועים יכולים להגיע מאוחר, offline, מ-CI, Claude, Codex, ומערכות שונות. ה-canonical envelope יכול, event identity, Run/Work identity, trace/span links, source sequence, repo revision ושלושה זמני lifecycle לפחות

(occurred, observed, ingested). המיפוי יכול להיות OpenTelemetry-compatible, אך EOS שומר namespace קנוני משלו כדי לא להיות תלוי ב־semantic conventions שעדיין נמצאים Development. OpenTelemetry מגדירה במפורש stability lifecycle ומזהירה מפני long-term dependencies על signals ש־Development.

Evidence isolation נגד self-confirming loops

```
origin_class:
  DEVELOPMENT
  QUALIFICATION
  OPERATIONAL
  HOLDOUT
  EXTERNAL_ATTESTATION
```

בנוסף:

```
independence_group
```

עשר ריצות כמעט זהות אינן עשר הוכחות עצמאיות. Dataset ששימש לבחור Champion לא מקבל אותה עוצמה כהוכחת holdout לכך שה־Champion טוב. זו נקודה קריטית במיוחד משום ש־evals עצמם יכולים להיות שבורים: OpenAI דיווחה בספטמבר 2026 על audit של SWE-Bench Pro שבו pipeline סימן % 27.4 מהמשימות כבעייתיות ו־human review זיהה % 1.4, עם failures כגון tests נוקשים מדי, coverage לא מספק.

Compatibility / Deprecation Contract

EOS — Asset, Project Profile, Telemetry, Evidence, Control, Agent Contract, כל חוזה ציבורי של Release Manifest ו־Simulation Manifest — מקבל:

```
schema_version
stability
introduced_in
deprecated_in
replacement
migration_path
```

Lifecycle בסיסי:

DEVELOPMENT → STABLE → DEPRECATED → UNSUPPORTED/REMOVED

Stable contract אינו מקבל breaking semantic change בתוך אותו major version. שינוי breaking דורש migration/version חדש. Unknown enum values צריכים להיות tolerated כאשר ניתן לעשות זאת בבטחה. מודל זה נשען על עיקרון stability דומה לזה של OpenTelemetry, שבו Stable שומר backward compatibility ו־deprecated replacement חייב להיות יציב לפני deprecation.

Release production. **Supply-chain contract.** אינה מזוהה רק בשם tag.

```
version
+ tag
+ source commit
+ artifact digest
+ provenance/attestation where supported
```

GitHub Immutable Releases נועלות את ה־tag ל־commit ואת release assets מפני החלפה לאחר Artifact attestations. publication מאפשרות לקשור artifact למקור ולאופן הבנייה, אבל GitHub מדגישה שה־attestation צריך גם לעבור verification ושעצם קיומו אינו הוכחה שהקוד בטוח.

לכן:

```
download
→ verify identity/digest
→ verify provenance if available
→ apply release policy
→ activate
```

Attestation הוא evidence של provenance, לא security score.

Resource / Overhead Budget. EOS נדרש להצדיק לא רק correctness אלא overhead:

startup latency
 resolve latency
 context tokens added
 tool calls added
 wall-clock overhead
 telemetry CPU/I/O
 Supabase storage/write growth
 eval cost
 human rescue time

אין מספרים קנוניים עדיין. Stage 0 מגדיר Stage 8 measurement; ו-4 1 מייצרים baseline; רק אז
 versioned budgets נקבעים. Regression משמעותי באחד הצירים אינו יכול להיעלם בתוך "overall score"
 יחיד.

Architecture Fitness Tests יהיו automated invariants, בין היתר:

Core must not import Claude-specific or Codex-specific semantics.
 Adapters must not own canonical knowledge semantics.
 Resolver must not mutate canonical Git.
 Asset Score must not consume raw telemetry directly.
 External registry must not become EOS S50T.
 Target project must not receive a full EOS runtime copy.
 Runtime must not silently fall back to latest unpinned release.
 Knowledge recommendation must not grant permission.
 A Solution Set must not expose multiple Champions.
 Critical Assurance must not fail open.
 Agent-visible content must not obtain instruction authority merely from provenance.

ההחלטות המאושרות

הטבלה הבאה היא ה-canonical summary של Decision 1–17. Dependencies מציינים תלות רעיונית, לא workflow waterfall.

D1 — Personal Engineering System

Statement, outcome ורציונל: המערכת מיועדת בראש ובראשונה למשתמש יחיד ולפרויקטים האישיים שלו.
 המטרה היא leverage בפיתוח, לא enterprise platform. כך נמנעים RBAC, tenancy, admin UI ו-governance
 ללא צורך מוכח.

Design + Contracts/Interfaces: Configuration מיינמלי; identities ברמת owner/project/run; user intervention רק כאשר consequential.

Failure modes + Telemetry/Evals: כשל: premature enterprise abstractions, setup, יותר זמן תחזוקה
 מערך. מדדים: EOS overhead, user rescue, setup time.

Dependencies + Migration/Compatibility: יסוד לכל השאר. Multi-user/team features נכנסים רק אם
 requirement אמיתי מופיע בעתיד.

D2 — Claude + Codex First, Agent-Neutral Core

Statement, outcome ורציונל: Claude ו-Codex הם agents ראשונים, אך domain core אינו בנוי סביב אף אחד מהם.

Design + Contracts/Interfaces: AgentAdapter; capability snapshot; common Agent Contract;
 agent/model/harness נרשמים ב-metadata.Run.

Failure modes + Telemetry/Evals: כשל: if claude בתוך domain logic, feature שזמין רק לאחד בלי fallback,
 מעבר model שדורש Eval. rewrite. אותה capability דרך שני agents.

Dependencies + Migration/Compatibility: תלוי D1; מזין Agent D13/D14. עתידי דורש adapter/negotiation, לא schema חדש.

D3 — Central Canonical Repository

Statement, outcome ורציונל: releases Core, schemas, canonical knowledge, Controls, eval definitions. נשמרים ב־repository קנוני אחד.

Design + Contracts/Interfaces: Stable IDs; versioned schemas; manifest; Promotion Path

Failure modes + Telemetry/Evals: כשל: שני SSOTs, copy-paste drift, assets שמתוקנים בפרויקט אחד ולא באחר. Eval: ownership/fitness tests.

Dependencies + Migration/Compatibility: בסיס D4/D6/D12. נכסי ה־EOS הישן נכנסים למודל החדש דרך D16, לא כ־fork של runtime.

D4 — Thin Project Footprint

Statement, outcome ורציונל: target project מחזיק רק מה שהוא באמת צריך: release pin, Project Profile. EOS לא עותק מלא של spec, ADRs/bootstrap metadata.

Design + Contracts/Interfaces: Installation manifest/pin; profile schema; optional minimal agent bootstrap.

Failure modes + Telemetry/Evals: כשל: project repos מתנפחים, divergence בין עותקי update EOS, ידני. Eval: fresh clone + restore. runtime ללא copied core.

Dependencies + Migration/Compatibility: D3; נשען בהמשך על D12. Compatibility דרך pinned release.

D5 — Dynamic Project Profile

Statement, outcome ורציונל: EOS מבין את הפרויקט באופן incremental: מסיק facts שניתנים לאימות, שואל רק על UNKNOWN consequential, ומבדיל intent מ־observed state.

Design + Contracts/Interfaces: ProjectProfile.spec; probes/status observations; Open Decision interface; provenance לכל fact.

Failure modes + Telemetry/Evals: כשל: silent guess; repeated onboarding; stale assumptions; mismatch שלא מזוהה. Eval: prototype→auth→production, declared/observed drift.

Dependencies + Migration/Compatibility: D4. מזין Existing projects ו־Resolver, Assurance, Project Fit D15. לא נדרשים ל־manual re-entry של facts שניתנים לגילוי.

D6 — Canonical Knowledge Asset Model

Statement, outcome ורציונל: Patterns, Templates, Skills, Reference Tests, Reference Repos, official docs/facts, Lessons, Failed Solutions, Provider/Integration knowledge. provenance ו־relationships.

Design + Contracts/Interfaces: Asset ID/schema; type; problem/capability; applicability; compatibility; provenance; lifecycle; freshness; risk; relationships; solution-set membership.

Failure modes + Telemetry/Evals: כשל: unstructured blobs, duplicate identity, Failed Solution
 כפתרון, source אבוד. Eval: schema/property tests, provenance completeness, corpus import
Dependencies + Migration/Compatibility: D3/D5. D7–D10
 מסתמכים עליו. Schema
 .versioned/migratable

D7 — Evidence-Based Catalog, Asset Score & Canonical Champion

Statement, outcome ורציונל: Assets מדורגים באמצעות Bayesian-style evidence model, אך Score, Project
 Cost ו Fit, Risk נפרדים. לאותה equivalence class מוצג Champion יחיד רק אם evidence מצדיק זאת.

Design + Contracts/Interfaces: Versioned scoring policy; evidence links; SolutionSet;
 .Champion/Challengers; quarantine override

Failure modes + Telemetry/Evals: כשל: score גבוה עם evidence קטן; correlated trials נספרים עצמאית;
 critical failure נבלע במוצג; שתי המלצות זהות מלפנים. Eval: evidence replay, independence groups.
 .Champion challenge/shadow

Dependencies + Migration/Compatibility: D6 + D11. Promotion
 לקנוני אינו רק max(score). משקלים/priors
 .versioned וניתנים לכיול.

D8 — Observation Curation Admission

Statement, outcome ורציונל: למידה מהעולם אינה משנה canonical knowledge ישירות. Observation הופך
 Candidate ורק לאחר dedupe, provenance, evidence qualification יכול להתקדם.

Design + Contracts/Interfaces: Observation schema; Curator; Candidate state; Promotion Proposal; risk-
 .based approval

Failure modes + Telemetry/Evals: כשל: self-modifying knowledge, poisoning, spam, duplicate
 proliferation, accidental authority escalation. Eval: malicious observation, duplicate, conflicting sources,
 .low-evidence promotion

Dependencies + Migration/Compatibility: D6/D7/D11
 מזין Canonical history D15. נשמר; supersession
 במקום overwrite אובדני.

D9 — resolve / inspect / expand Knowledge Contract

Statement, outcome ורציונל: interface קטן ויציב בין agents לידע. resolve מחזיר מעט ורלוונטי; inspect
 מעמיק; expand מחפש מעבר לברירת המחדל.

Design + Contracts/Interfaces: Request/response schemas; handles/IDs; relevance/project-fit metadata;
 .transport-neutral semantics

Failure modes + Telemetry/Evals: כשל: context dump, low recall, high noise, direct provider routing,
 .duplicate champions. Eval: recall, precision, context size, latency, missed critical asset

Dependencies + Migration/Compatibility: D5–D7
 נמסר דרך Contract Stable D13. עם
 .migration/versioning

D10 — Adaptive Assurance

Statement, outcome ורציונל: Controls מופעלים לפי Project Profile, risk evidence; אינם workflow policing. Critical gates יכולים לחסום רק כאשר consequence מצדיק זאת.

Design + Contracts/Interfaces: Control schema; applicability rules; states כגון .SATISFIED/MISSING/UNKNOWN/EXEMPT; revision-bound evidence; expiring exemptions

Failure modes + Telemetry/Evals כשל: כל project מוצף ב־ UNKNOWN→SATISFIED; production controls; stale test proves new HEAD; critical gate fail-open. Eval: project-state progression, stale evidence, exemptions, negative controls

Dependencies + Migration/Compatibility: D5/D6/D11. NIST SSDF תומך באינטגרציה של security outcomes לתוך SDLC בלי להכתיב lifecycle יחיד.

D11 — Telemetry, Evidence & Investigation

Statement, outcome ורציונל: metadata-first telemetry נאסף אוטומטית, נשמר locally במקרה של outage, נשלח ל־Evidence Plane ומומר ל־Evidence באמצעות Raw event derivers versioned. Evidence אינו בפני עצמו.

Design + Contracts/Interfaces: Work/Run/Event IDs; causal envelope; SQLite outbox; Supabase ingestion; evidence derivers; failure clustering; Investigation View

Failure modes + Telemetry/Evals כשל: secrets/prompts נשמרים; duplicate/out-of-order events; crash נראה Supabase outage success; exposed asset coding; credit כאלו applied. Eval: offline, crash, partial sync, delayed CI, rework, replay

Dependencies + Migration/Compatibility: D5–D8. תומך Schema D7/D10/D14. צריך להיות OTel-compatible EOS-owned; MDגישה stability/versioning לחוזים ארוכי-חיים. אך OTel

D12 — Pinned Releases, Launcher, Restore & Rollback

Statement, outcome ורציונל: פרויקט רץ מול release מדויק, לא tiny launcher latest; משחזר, מאמת, מעדכן ומחזיר אחורה באופן transaction-like

Design + Contracts/Interfaces: installation.json/manifest; version cache; doctor; restore/update/rollback; artifact digest/provenance

Failure modes + Telemetry/Evals כשל: machine חדש לא משחזר; cache; partial update; tampered release; corruption; project A מתחלף כי kill update, rollback, tamper. Eval: clean machine, offline cache, B updated.

Dependencies + Migration/Compatibility: D3/D4. GitHub Immutable Releases מספקות מנגנון להגנת tag/assets; attestations רק כאשר מאמתים אותן.

D13 — Future-Adaptive Agent Integration

Statement, outcome ורציונל: Agents בוחרים בעצמם plugins/MCP/native/CLI מתוך מה שבאמת זמין ומורשה. EOS מספק context/evidence/constraints ולא hard-routing. ככל שה־AI משתפר, adapters מאצילים יותר.

Design + Contracts/Interfaces: Agent Contract; capability discovery; runtime snapshot; thin adapters; MCP .+ CLI/native delivery options

agent ל־model-specific core; EOS tool X hard-coded: כשל: **Failure modes + Telemetry/Evals** permissions. Eval: divergent toolsets, integration failure, recommendation משנה; מתקדם לעבוד פחות טוב; .future capability unknown to EOS

Dependencies + Migration/Compatibility: D2/D9/D12/D15. MCP transport נשאר; השינוי המשמעותי של spec ביולי 2026 מדגים למה לא לקשור domain state לסשנים/protocol details.

D14 — Verification, Evals & Qualification

layered proof: deterministic tests → composition/fault injection → agent evals **Statement, outcome** ורציונל: → controlled comparisons → real-project qualification → longitudinal evidence

Design + Contracts/Interfaces: Eval/Simulation manifests; hidden evaluator workspace; grader hierarchy; .capability/regression/holdout banks; harness snapshot

broken eval, reward hacking, one stochastic run, LLM judge כשל: **Failure modes + Telemetry/Evals** known-good/known- עם eval של ה־eval treated as truth, harness difference misattributed to EOS. Eval .bad/mutations

evals מגדירה OpenAI. מאמת את כל המערכת. D9–D13 תלוי **Dependencies + Migration/Compatibility** כבדיקות מובנות למערכות nondeterministic וממליצה על traces/graders/datasets עבור agents. Evals remain "as code"; ממשיך Evals הנוכחי של OpenAI עצמו מתוכנן להיסגר בנובמבר 2026 ולכן הוא אינו SOT שלנו.

D15 — Dynamic External Ecosystem

Runtime Connection. External Provider, Integration מ־Capability ורציונל: **Statement, outcome** .live verification knowledge freshness-aware; consequential volatile facts

Design + Contracts/Interfaces: capability graph; provider/integration assets; discovery adapters; freshness .classes; live overlay; runtime health/authorization

MCP registry treated trusted; stale pricing/provider fact; MCP כשל: **Failure modes + Telemetry/Evals** becomes core; provider and connector conflated; catalog tries index entire internet. Eval: new native tool, .stale fact, deprecation, registry risk, provider-vs-tool decision

metadata/discovery layer הרשמי הוא **Dependencies + Migration/Compatibility:** D6/D8/D13. MCP Registry ואף מצהיר שהוא unopinionated וש־security scanning delegated

D16 — Reuse Knowledge, Rebuild System Cleanly

Statement, outcome ורציונל: כל נכסי הידע הקיימים הם input חשוב לחדש; implementation הישן הוא .runtime base ולא reference/lessons בלבד

Design + Contracts/Interfaces: one-time/controlled importer into Asset model; provenance mapping; .dedupe; classification; no old-system runtime dependency

Failure modes + Telemetry/Evals: כשל: להתחיל מאפס ולאבד ידע; או refactor של המערכת הישנה שסוחר
 Eval: full corpus accounted for, useful assets import, old policies not resurrected, governance והנחות ישנות. duplicate handling
Dependencies + Migration/Compatibility: D3/D6–D8. Legacy IDs/provenance נשמרים כשאפשר;
 implementation compatibility אינה יעד.

D17 — Simulation-Gated Implementation & Release

Statement, outcome ורציונל: בונים smallest complete system, מוכיחים כל claim בסימולציה מציאותית, ורק
 אז מסתמכים עליו. Stages הם proof gates, לא waterfall.

Design + Contracts/Interfaces: Simulation manifest; stage gates; release rings; recovery/rollback; real
 canaries; shadow mode

Failure modes + Telemetry/Evals: כשל: "code complete" נחשב done; success criteria משתנים לאחר
 bureaucracy. Eval: gate validity, fresh failure; build-all-first; Project 8 custom-coached; gate
 environments, fault cases, reproducibility

Dependencies + Migration/Compatibility: תלוי בכל D1–D16. Work יכול לחפוף בין stages; promotion
 cannot

מפת התלויות העיקרית:

```

flowchart LR
    D1["D1 Scope"] --> D2["D2 Agent-neutral"]
    D1 --> D3["D3 Canonical Repo"]
    D3 --> D4["D4 Thin Projects"]
    D4 --> D5["D5 Project Profile"]
    D3 --> D6["D6 Assets"]
    D6 --> D7["D7 Evidence/Champion"]
    D6 --> D8["D8 Curation"]
    D5 --> D9["D9 Resolver"]
    D6 --> D9
    D5 --> D10["D10 Assurance"]
    D6 --> D10
    D11["D11 Telemetry/Evidence"] --> D7
    D11 --> D8
    D11 --> D10
    D3 --> D12["D12 Releases"]
    D2 --> D13["D13 Agent Integration"]
    D9 --> D13
    D12 --> D13
    D13 --> D15["D15 External Ecosystem"]
    D8 --> D15
    D9 --> D14["D14 Evals"]
    D10 --> D14
    D11 --> D14
    D13 --> D14
    D6 --> D16["D16 Reuse Assets"]
    D17["D17 Simulation Gates"] --> D14
    D1 --> D17
  
```

החצים כאן מתארים semantic dependency בלבד; הם אינם קובעים שכל שורת קוד בשלב מוקדם חייבת
 להסתיים לפני שניתן לעבוד במקביל על רכיב מאוחר יותר.

תוכנית המימוש מבוססת הסימולציות

כל Simulation היא artifact versioned, ובכל Stage יש לפני promotion:

Claim
 Environment

Project A/vA + Project B/vB; kill update; corrupt cache; fail migration; remove network. Hidden: tampered artifact ואחד manifests מצביע לגרסה שונה. Intervention: רק commands ציבוריים
 Success: exact versions; no silent latest; failed update leaves (install/doctor/restore/update/rollback) known-good state; doctor identifies fault; rollback works. Failure: half-install, cross-project version drift, unsigned/tampered activation. Evidence: release identity, digest verification, state transitions. Recovery: rollback/re-download known-good artifact. Repetition: OS/runtime combinations supported. Scenario: laptop חדש משחזר project ישן מה־repo וה־release metadata.

Stage 2 — Existing EOS Asset Import

Environment: architecture הישן. **Qualification Manifest:** Claim מתחילים מהידע הקיים בלי לייבא את Start: empty new Asset store. Stimulus: full import. importer חדש. corpus אמיתי של snapshot
 Hidden: duplicate Patterns, overlapping Skills, Failed Solution, stale doc, weak provenance, old governance intervention כל; ambiguous באמת classification metadata. Intervention: manual review
 Success: כל useful knowledge asset accounted for; provenance נשמר; old מסומן/מסומן unknown; runtime rule; duplicates grouped; no unsupported canonical winner. Failure: workflow policy לא נהיה
 silent deletion, fake provenance, mandatory old policy resurrected. Evidence: inventory-before/after, mapping report, duplicate groups. Recovery: importer idempotent; reset and replay. Repetition: full import
 עד Scenario: stable mapping; regression fixture afterward. auth patterns מהישן מתגלים כשני duplicates ופתרון אחד context שונה.

Stage 3 — Project Profile & Onboarding

guess. Environment: existing real- ובלי incremental באופן פרויקט מבין **Qualification Manifest:** Claim: EOS like repos עם facts ידועים/לא ידועים. Start: minimal pin; profile. ריק. Stimulus: probes + normal agent
 Intervention: user onboarding. Hidden: conflicting provider configs; production state repo. Success: verifiable facts inferred; unknown מה־ לפתור אפשר עונה רק לשאלות
 remains UNKNOWN; declared/observed mismatch becomes drift; no repetitive onboarding. Failure: silent guess, needless questions, status overwrites spec. Evidence: fact provenance, probe trace, user-question count. Recovery: remove derived status and rescan. Repetition: multiple archetypes. Scenario: deployment workflow פעיל. production=false declared

Stage 4 — Knowledge Registry + Resolver

Environment: משמעותי. **Qualification Manifest:** Claim המערכת יכולה למצוא מעט ידע נכון מתוך corpus Start: validated Project Profile. Stimulus: backend bug, CLI בלבד, imported corpus + task bank
 auth, DB migration, upload, testing, mobile persistence. Hidden: expected critical assets + deliberately irrelevant near-matches. Intervention: none during scoring. Success: high critical recall, acceptable precision/context/latency, one Champion per equivalent set, inspect fetches detail only on demand, expand handles missing default result. Failure: full catalog dump, missed critical asset, Failed Solution recommended, duplicate Champions. Evidence: retrieval metrics, selected IDs, latency/context size. Recovery: index rebuild. Repetition: fixed regression bank + holdout tasks. Scenario: authentication task
 .returns canonical auth pattern and relevant Control, not five auth assets

Stage 5 — Telemetry Foundation

Qualification Manifest: Claim: events survive failures without leaking sensitive content or creating duplicates. Environment: local runtime + SQLite + test Supabase plane. Start: no records. Stimulus: normal runs, offline mode, duplicate batches, process crash, partial network acknowledgement, secret-like payload. Hidden: one crash before Stop; same event delivered three times. Intervention: none except restoring network. Success: durable local queue; idempotent remote records; interrupted run identified; sanitization blocks disallowed payload; partial retry safe. Failure: lost events, duplicate evidence basis, raw secret persisted, coding blocked solely because remote telemetry is down. Evidence: event IDs, outbox before/after, remote row identity, redaction tests. Recovery: local replay; corrupted batch quarantined. **offline עובד** Repetition: deterministic fault suite. Scenario: agent **מאוחזר יותר בלי**.duplication

Stage 6 — Evidence + Investigation

Qualification Manifest: Claim: raw telemetry **ניתן להמרה ל־Evidence explainable** ולתחקור אמין. Environment: known synthetic and real-like traces. Start: telemetry from Stage 5. Stimulus: exposed/inspected/applied assets, test catching bug, retries, late CI failure, strategy change. Hidden: ground-truth causal timeline. Intervention: none. Success: attribution differentiates exposure/use/outcome; late CI updates evidence; rework/recurrence detected; Investigation does not invent causality. Failure: asset receives success because merely displayed; timestamp ordering treated causal; delayed failure ignored. Evidence: derivation version, source links, expected-vs-generated timeline. Recovery: replay raw retained telemetry with deriver revision. Repetition: every deriver change over golden fixtures. Scenario: CI failure **יום אחר**.Run reclassifies evidence linked to same revision

Stage 7 — Agent Contract + Adapters

Qualification Manifest: Claim: same semantics work through more than one delivery mechanism and agent-specific adapters remain thin. Environment: MCP/CLI + agent adapter test harness. Start: resolver/evidence operational. Stimulus: same calls over MCP and CLI; MCP dies; native Skill unavailable; read-only connection. Hidden: one integration advertises capability but lacks authorization. Intervention: agent may select available fallback. Success: semantic parity; graceful degradation; no permission escalation; no core branch based on agent name. Failure: transport changes meaning; unavailable Skill blocks access to knowledge; recommendation grants write. Evidence: contract traces, capability snapshots, permission decisions. Recovery: CLI/native fallback. Repetition: per adapter/release. Scenario: inspect(asset) **returns equivalent semantic object through CLI and MCP**

Stage 8 — First Real Agent Vertical Slice

end-to-end. **Qualification Manifest:** Claim: agent **אמיתי מסוגל להשתמש ב־EOS בצורה טבעית** end-to-end. Environment: disposable but realistic repo; current primary agent; fresh session. Start: pinned EOS + healthy telemetry. Stimulus: ordinary engineering task prompt; one misleading clue; test failure during work. Hidden: no instruction saying to call resolve; expected outcome known to evaluator only. Intervention: user may answer genuine architectural decision; no EOS coaching. Success: agent discovers/uses EOS as useful; produces correct code/tests; telemetry→evidence→investigation complete; no rescue. Failure: user must remind “use EOS”; EOS advice degrades solution; telemetry gap obscures outcome. Evidence: repo diff/tests, trace, tool choices, retrieved assets, investigation. Recovery: disposable reset. Repetition: several independent trials/tasks. Scenario: bug fix requiring a previously imported lesson **from old EOS**

:real-world qualification intelligence, robustness 70 Stages

Stage 9 — Adaptive Assurance

Qualification Manifest: Claim: assurance increases with consequence without returning to workflow policing. Environment: same repo represented as prototype/auth/production/destructive-change states. Start: working vertical slice. Stimulus: add auth, enable production, change security-sensitive revision, create exemption. Hidden: stale evidence + unknown critical fact. Intervention: user can approve a legitimate exemption/decision. Success: only applicable Controls surface; evidence tied to revision; UNKNOWN remains UNKNOWN; exemption expires; destructive critical state fail-closes. Failure: prototype flooded; stale evidence accepted; ordinary workflow constrained unnecessarily. Evidence: Control state transitions and reason graph. Recovery: revert project state/decision. Repetition: per Control engine .revision. Scenario: auth code changes after security test; prior SATISFIED becomes stale

Stage 10 — Learning, Asset Scores & Canonicalization

Qualification Manifest: Claim: accumulated evidence improves ranking without creating false certainty or duplicate visible solutions. Environment: controlled runs with known evidence dependencies. Start: competing candidates. Stimulus: positive/negative/partial outcomes, 500 correlated repeats, one critical failure, new independent holdout. Hidden: correlated trials share same root cause. Intervention: human only for consequential promotion dispute. Success: deterministic score replay; correlation discounted; critical failure can quarantine; Champion changes only after sufficient independent evidence; distinct contexts remain separate. Failure: popularity=truth; high numeric score bypasses quarantine; two Champions. Evidence: score explanation, origin/independence groups, promotion history. Recovery: recompute from source Evidence under previous policy. Repetition: property tests + periodic Champion .challenges. Scenario: old champion loses to challenger after several independent production outcomes

Stage 11 — External Ecosystem

Qualification Manifest: Claim: system consumes current ecosystem knowledge without trusting/discovering/installing conflation. Environment: official docs, MCP registry, provider adapters, simulated stale cache. Start: canonical baseline intentionally older than one fact. Stimulus: new integration, stale pricing, provider deprecation, registry entry with weak security evidence. Hidden: discovered server is authentic publisher but unsafe candidate. Intervention: user only for provider/architecture commitment or new high-risk permission. Success: live verification for volatile consequential fact; registry≠trust; deprecation becomes Observation; new native capability usable without core rewrite; connector choice remains agent-owned. Failure: catalog fact silently overwrites baseline; discovered MCP auto-installed; Supabase-vs-Firebase silently chosen. Evidence: source/retrieved_at, trust vector, runtime choice. Recovery: discard overlay/candidate, fall back to pinned baseline where safe. Repetition: provider adapters + .freshness cases. Scenario: current pricing differs from cached Provider Asset during architecture decision

Stage 12 — Multi-Agent & Capability Dynamics

Qualification Manifest: Claim: Claude, Codex and changing capabilities share the same OS semantics without identical tools. Environment: two agents with intentionally different integrations. Start: same Work Item/repo SHA. Stimulus: one uses MCP, one native/CLI; handoff; concurrent work; simulated new capability. Hidden: preferred tool fails for one agent. Intervention: no instruction on tool choice. Success: both complete via different paths; handoff preserves Work/Evidence; concurrency does not corrupt EOS state; unknown capability negotiated gracefully. Failure: hard route, agent-specific knowledge semantics,

history loss. Evidence: capability snapshots, selected integrations, outcomes/handoffs. Recovery: isolate Run state/replay. Repetition: both agents + future adapter qualification. Scenario: Claude implements, Codex .reviews/fixes from same Work Item

Stage 13 — Resilience, Security & Scale

Qualification Manifest: Claim: EOS fails safely and predictably under infrastructure, malicious content and scale stress. Environment: fault injection sandbox + synthetic large stores. Start: stable multi-agent build. Stimulus: Supabase outage, SQLite lock/full disk, prompt injection in external content, release tampering, concurrent runs, 100k assets/10k controls/1m telemetry records. Hidden: one external doc contains malicious instruction text. Intervention: only documented recovery operations. Success: no corruption; no false completeness; external data has no instruction authority; release mismatch blocked; critical gate fail-closed; performance within provisional budgets. Failure: secret/authority leak, data loss, malicious content controls execution, silent incomplete evidence. Evidence: fault logs, invariants, performance metrics, integrity checks. Recovery: local queue, rollback, restore, quarantine. Repetition: critical faults every RC; scale periodically. Scenario: Supabase unavailable during coding while destructive release decision .simultaneously needs unverifiable evidence

Stage 14 — Native vs Assisted, Ablation & Shadow

Qualification Manifest: Claim: EOS adds measurable value and does not merely preserve legacy complexity. Environment: controlled pairs with same model/task/tools/budget/environment; shadow variants. Start: feature-complete candidate. Stimulus: native-only vs EOS-assisted trials; remove Skills/Assurance/history individually; Resolver v2 shadow. Hidden: holdout tasks not used to tune EOS. Intervention: none except genuine project decisions. Success: no critical quality regression; useful improvement in target dimensions such as fewer misses/rework/rescues at acceptable overhead; redundant subsystem identified when applicable. Failure: EOS lowers quality or adds large overhead without benefit; shadow misses critical knowledge. Evidence: vector metrics, trial distributions, cost/context/tool counts. Recovery: retain previous active subsystem; shadow candidate discarded. Repetition: model/major EOS changes. Scenario: new agent model already does testing strategy natively; EOS Test Guidance is removed if .no longer beneficial

Stage 15 — Project 8 Canary

Qualification Manifest: Claim: system works on genuine work rather than tasks designed around EOS. Environment: real Project 8, clean worktree, known SHA, pinned candidate release, fresh agent session. Start: known baseline tests + rollback. Stimulus: at least two useful independent bounded tasks. Hidden: no EOS-specific instructions and normal project ambiguities remain. Intervention: only expected architectural/user decisions; rescue counts negative. Success: real outcome correct, repo CI/tests pass, EOS use natural or correctly unnecessary, no unexpected rescue, full evidence/investigation. Failure: success depends on coaching, task crafted around known EOS asset, project-specific assumptions in core. Evidence: real diff/CI/telemetry/evidence/user interventions. Recovery: git/release rollback. Repetition: ≥ 2 materially different tasks before promotion. Scenario: database/backend change followed by unrelated debugging .task

Stage 16 — Cross-Project Qualification

Qualification Manifest: Claim: architecture generalizes beyond Project 8. Environment: second materially different real project; SportReel is current candidate because mobile/video/cloud/AI differs from

backend/database. Start: fresh project integration, same EOS release. Stimulus: bounded real task, different tool environment, possible reuse of migrated old asset. Hidden: assumptions that only worked in Project 8. Intervention: normal project decisions only. Success: different resolver results/Controls/integrations work without core changes; imported knowledge can acquire fresh evidence. Failure: hard-coded Project 8 paths/provider assumptions. Evidence: cross-project metrics and config delta. Recovery: project-specific rollback. Repetition: multiple archetypes over time, not dozens initially. Scenario: SportReel task uses .cloud/video/mobile constraints absent from Project 8

Stage 17 — RC -Stable

Qualification Manifest: Claim: exact artifact intended for stable use survives installation, upgrade, rollback and real qualification. Environment: clean machine + old installation + release infrastructure. Start: 1.0.0-rc.x artifact. Stimulus: install from artifact, upgrade old pinned project, verify digest/provenance, fail a canary, rollback, then successful ring progression. Hidden: tampered artifact / one failed migration candidate. Intervention: release halt/rollback allowed. Success: artifact identity matches source/release manifest; restore/update transactional; all critical suites pass; real rings show no critical regression. Failure: “tested source” differs from shipped artifact, rollback fails, canary negative but promotion continues. Evidence: release digest, attestations where available, suite results, canary metrics, rollback proof. Recovery: previous immutable release. Repetition: every RC/release. Scenario: RC succeeds in .synthetic/shadow, fails Project 8 and is automatically held instead of becoming stable

scale משתמש במספרי scale כ־stress margin, לא כ־business requirement. לפני הרצת scale qualification Stage 0/8 יקבעו provisional latency/storage budgets כדי למנוע מצב שבו “PASS” מוגדר בדיעבד.

כל עובר רק אם:

Claim proven — ה־claim של השלב הוכח, לא רק code completion

Fresh environment — אין hidden state לא מתועד

Positive case — happy path עבר

Negative/fault case — הכשל החשוב נבדק

Evaluator validity — known-good ו־known-bad מוזוהים

Telemetry/evidence complete — ניתן להסביר את התוצאה

No silent guessing — UNKNOWN נשאר מפורש

Recovery proven — לכל stateful/high-risk stage

Reproducibility — deterministic replay או multiple stochastic trials

Relevant regression bank — אין critical regression

כשירות, תפעול ושחרור

Qualification architecture משתמשת בשש שכבות:

```

Deterministic software tests
↓
Composition / contract tests
↓
Fault & adversarial simulations
↓
Agent behavioral evals
↓
Controlled / shadow comparisons

```

↓
Real-project qualification
↓
Longitudinal operational evidence

OpenAI ממליצה על evals מובנים למערכות nondeterministic ועל trace/graders/datasets עבור agent workflows. העובדה שה־Evals platform הנוכחית של OpenAI עצמה נמצאת במסלול deprecation — read-only ב־1 3 באוקטובר 2026 וסגירה מתוכננת ב־0 3 בנובמבר 2026 — מחזקת את ההחלטה לשמור את manifests וה־graders של Improved-Engineering-OS כ־code/data קונוניים משלנו, עם adapters לשירותי צד שלישי ולא להפך.

:Grading hierarchy

Deterministic outcome/state
>
Static/type/security checks
>
Trace/trajectory rules
>
LLM grader
>
Human calibration/review

LLM grader מתאים לשאלות כמו "האם ההמלצה הייתה רלוונטית?", אבל לא צריך לגבור על test שמראה שהקוד נכשל.

כל grader קריטי מקבל:

Positive control
Negative control
Mutation case where relevant

Agent workspace ו־evaluator workspace מופרדים; hidden tests/expected outputs אינם נגישים לסוכן.

:Eval banks

Regression — יכולת שכבר נדרשת ולא אמורה להישבר

Capability/Challenge — גבול היכולת שאנחנו מנסים לשפר

Holdout — זיהוי overfitting/eval gaming

Operational Replay — failures/successes שנבעו מעבודה אמיתית

Adversarial — poisoning, permission, prompt injection, destructive behavior

Capability scenario שנעשה trivial בגלל AI מתקדם אינו נשאר לנצח "benchmark חשוב"; הוא עובר ל־Regression smoke או מוסר אם כבר אין signal.

:Native-vs-Assisted הוא חלק קונוני מהתחזוקה.

Same agent + same task + same tools + same budget
↓
Native
vs
EOS-assisted

ההשוואה מיועדת לשאלה "מה התרומה האינקרמנטלית של EOS?". בנוסף תבוצע realistic system comparison שבה כל setup יכול להשתמש ביכולות native האידיאליות שלו; זו שאלה אחרת: "באיזו מערכת באמת כדאי לעבוד?".

אין Overall Score יחיד. Dashboard release צריך להציג וקטור כגון:

task success
critical knowledge recall
knowledge precision
critical-control misses
false-positive controls
rework

unexpected user rescue
tokens/context
tool calls
wall-clock time
resolve latency
telemetry completeness
infra-error rate
EOS/storage cost

שיפור tokens אינו יכול "לבטל" regression באבטחה, ולהפך.

Evidence attribution מבדיל לפחות:

EXPOSED
INSPECTED
APPLIED
DIRECTLY_VERIFIED
FAILED / CONTRIBUTED_TO_FAILURE

Asset לא מקבל success evidence רק כי הופיע ב־resolve.

Failure taxonomy

TASK_FAILURE
AGENT_FAILURE
EOS_FAILURE
INTEGRATION_FAILURE
AUTHORIZATION_FAILURE
PROVIDER_OUTAGE
EVALUATOR_FAILURE
INFRA_ERROR

אין rerun-until-pass שמוחק attempts קודמים.

Release Rings

Development

Exposure: local/unit

מטרת השער: correctness בסיסי

Rollback: trivial

Disposable Simulation

Exposure: isolated realistic env

מטרת השער: integration/fault behavior

Rollback: reset sandbox

Shadow

Exposure: real inputs, no behavioral effect

מטרת השער: compare new resolver/score/control

Rollback: disable shadow

Single-task Canary

Exposure: Project 8 bounded task

first real impact: **מטרת השער:**

Rollback: previous release + git rollback

Multi-task Canary

Exposure: כמה tasks עצמאיים

reproducibility: **מטרת השער:**

Rollback: same

Cross-project Canary

Exposure: project archetype נוסף

generalization: **מטרת השער:**

Rollback: project-specific rollback

RC

Exposure: shipped artifact

exact artifact qualification: **מטרת השער:**

Rollback: previous immutable release

Stable

Exposure: default approved release

ongoing real work: **מטרת השער:**

Rollback: pinned previous stable

Release identity חייב להיות immutable/verifiable ככל האפשר. GitHub Immutable Releases נועלות tag ו־assets לאחר publication. כאשר artifact attestations זמינות ומתאימות ל־repository/plan, הן ישמשו ל־provenance verification; GitHub artifact attestation כטענה שה־artifact עצמו נקי מפגיעויות. לכן ה־contract הוא provider-neutral: digest + immutable identity הם baseline; cryptographic build provenance נוסף כאשר התשתית תומכת בכך.

Telemetry/Data Flow

flowchart LR

```

A["Agent / Tool / CI"]
R["EOS Runtime"]
Q["SQLite Durable Outbox"]
T["Supabase Raw Telemetry"]
D["Versioned Evidence Deriver"]
E["Evidence"]
I["Investigation / Failure Clusters"]
S["Asset Scores / Project Fit"]
O["Observations"]
C["Curator"]
P["Promotion Candidate"]
G["Canonical Git"]

```

```
REL["Future Release"]
```

```
A --> R
R --> Q
Q --> T
T --> D
D --> E
E --> I
E --> S
I --> O
S --> O
O --> C
C --> P
P --> G
G --> REL
```

Graceful degradation: remote telemetry outage אינו אמור לעצור coding שאינו תלוי בו; ה־outbox המקומי סופג אותו. לעומת זאת, פעולה destructive שה־Assurance שלה דורש evidence שלא ניתן כרגע לאמת אינה מקבלת status ירוק רק כדי "להמשיך לעבוד".

External tool discovery נשאר 2026-07-28 MCP context-sensitive. כולל cacheable list behavior ועבר לליבה stateless, מה שמאפשר adapters לבצע discovery/caching בלי להפוך session state לחלק מה־Anthropic domain model. כבר מממשת deferred tool discovery בפלטפורמה שלה, דבר שמחזק את הכיוון של לחשוף רק את מה שרלוונטי ולא מאות tool definitions upfront.

Supply-chain/security qualification חייבת לכלול גם Improved-Engineering-OS עצמו, לא רק את הפרויקטים שעליהם הוא עובד. NIST SSDF מגדיר secure development practices כפעילות שצריכה להשתלב ב־SDLC ולהתמודד גם עם root causes ו־recurrence של vulnerabilities. ב־IEOS זה מתורגם ל־threat model של boundaries, verified releases, malicious external content tests, credential separation, provenance וחזרה בטוחה לגרסה ידועה.

Configuration scopes יהיו:

```
EOS Release Defaults
  ↓
Optional Owner Defaults
  ↓
Project Profile spec
  ↓
Current Run constraints
```

scope מקומי יכול לצמצם behavior אך אינו משנה בשקט החלטה ארכיטקטונית Prompt durable. חד־פעמי "נסה Firebase" אינו מחליף database_provider ב־Project Profile ללא החלטה מפורשת.

נספחים וצעדים הבאים

Asset Schema — conceptual baseline

```
schema_version: "1"

id: "asset.<stable-id>"
type: "pattern | skill | template | reference_test | lesson | failed_solution | provider | integration | ..."

title: "..."
status: "candidate | active | superseded | quarantined | deprecated"

problem:
  id: "problem.<id>"
  capabilities:
    - "database.schema.read"

solution_set_id: "solution-set.<id>"

applicability:
  conditions: []
```

```

compatibility:
  platforms: []
  providers: []
  constraints: []

provenance:
  - source_type: "official_docs | repo | existing_eos | observation | ..."
    source_identity: "..."
    source_revision: "..."
    observed_at: "..."
    integrity: "verified | partial | unknown"

freshness:
  class: "stable | normal | volatile | live_required"
  last_verified_at: "..."

risk:
  execution_authority: "data_only | executable"
  blast_radius: "read_only | write | destructive | privileged"

relationships:
  supersedes: []
  superseded_by: []
  related_to: []

evidence_policy:
  eligible_origins: []

```

Solution Set, מִנְגֵּר state הוא Champion. author-controlled שדה canonical: true
Evidence policy.

Project Profile — conceptual baseline

```

schema_version: "1"
project_id: "..."

eos:
  pinned_release: "..."
  release_digest: "..."

spec:
  lifecycle: "prototype | internal | production | ..."
  database_provider: "supabase | ..."
  authentication_required: true
  deployment: "..."
  architecture_constraints: []

decisions:
  - decision_id: "ADR-..."
    subject: "database_provider"

known_unknowns:
  - "production_user_count"

# observed/status is not continuously committed here.
# Runtime observations live in the Evidence Plane and are compared with spec.

```

Telemetry Envelope — conceptual baseline

```

schema_version: "1"

event_id: "uuid"
event_type: "..."

project_id: "..."
work_id: "..."
run_id: "..."

trace:
  trace_id: "..."
  span_id: "..."
  parent_span_id: null
  links: []

```

```

time:
  occurred_at: "..."
  observed_at: "..."
  ingested_at: "..."

source:
  type: "agent | tool | ci | eos"
  sequence: 42

revision:
  repo_sha: "..."
  eos_release: "..."

harness:
  agent: "..."
  model: "..."
  adapter_version: "..."
  available_capabilities_hash: "..."

attributes:
  # metadata-first, schema-controlled, sanitized
  key: "value"

```

.default attributes ותוכן Raw prompts, secrets אינם

Evidence Schema — conceptual baseline

```

schema_version: "1"

evidence_id: "..."
subject:
  type: "asset | control | integration | project | ..."
  id: "..."

kind: "success | failure | partial | verification | rework | ..."

origin_class: "development | qualification | operational | holdout | external_attestation"
independence_group: "..."

strength:
  polarity: "positive | negative | neutral"
  weight: 0.0
  confidence: 0.0

attribution:
  exposure: "exposed | inspected | applied | directly_verified"
  source_event_ids: []
  trace_id: "..."

revision:
  repo_sha: "..."
  deriver_version: "..."

lifecycle:
  created_at: "..."
  stale_after_revision_change: true
  expires_at: null

```

Simulation Manifest — conceptual baseline

```

schema_version: "1"

id: "simulation.<id>"
version: 1

claim: "What exactly this simulation proves"

environment:
  repo_revision: "..."
  eos_release: "..."
  agent: "..."
  model: "..."
  tools: []
  permissions: []
  network_policy: "..."

```

```

    budgets: {}

starting_state: {}

stimulus:
  task: "... "

hidden_conditions_ref: "evaluator-only://..."

allowed_interventions:
  user_architecture_decision: true
  eos_coaching: false

success_criteria: []
failure_criteria: []
stop_conditions: []

graders:
  deterministic: []
  trace_rules: []
  model_graders: []

validity_controls:
  positive: []
  negative: []
  mutation: []

required_evidence: []

recovery:
  procedure: "... "

repetition_policy:
  type: "deterministic | stochastic"
  minimum_trials: null
  stopping_rule: "versioned policy"

```

Glossary

Asset — יחידת ידע reusable ומזוהה עם provenance

Solution Set — קבוצת פתרונות לאותה equivalence class

Champion — פתרון יחיד שמוצג כברירת מחדל לאחר evidence מספיק

Challenger — פתרון חלופי שנשמר לצורך comparison/עמיד

Project Fit — מידת התאמה לפרויקט הנוכחי; נפרד מאיכות גלובלית

Observation — מידע שנצפה אך טרם הוכנס ל־canonical knowledge

Evidence — טענה נגזרת ומקושרת למקור/outcome שניתן לתחקור

Control — דרישת Assurance conditionally applicable

Work Item — יחידת עבודה durable שעשויה לכלול כמה Runs/agents

Run — execution episode מסוים עם harness/capabilities מוגדרים

Capability — מה צריך לבצע, בלי להניח מי הספק

Provider — service/technology המספק capability

Integration — דרך טכנית לגשת ל־provider/capability

Runtime Connection — integration שבאמת זמינה/healthy/authorized ב־Run

Pinned Baseline — גרסת EOS קבועה שניתנת לשחזור

Live Overlay — facts עדכניים ומאומתים לזמן הנוכחי שאינם משנים baseline

Curator — מנגנון שממיר Observations ל־Candidates ומיישם admission rules

Qualification — הוכחה שהמערכת מתאימה לשימוש, מעבר ל־unit tests

Shadow — גרסה/logic שרצים על inputs אמיתיים בלי להשפיע על execution

Canary — exposure אמיתי מוגבל לפני promotion

Architecture Fitness Test — בדיקה שאוכפת boundary ארכיטקטוני לאורך זמן

מפת יישום מועדפת, לפי עדיפות

P0 — להקפיא את החוזים, לא את המימוש. ליצור repository חדש Improved-Engineering-OS, להכניס את Architecture Constitution, state-ownership matrix וה-conceptual schemas של Stage 0. להגדיר namespace/versioning convention ו-Simulation Manifest. אין צורך עדיין ב־UI, semantic search, daemon או provider marketplace.

P0 — לבנות Simulation Harness לפני feature sprawl. הוא צריך לאפשר sandbox נקי, evaluator workspace נפרד, artifact collection, deterministic fixtures ו־Stage 0 multiple agent trials. infrastructure צדדי; הוא מנגנון ההוכחה לכל הפרויקט.

P0 — לבצע inventory מלא של נכסי Engineering OS הקיימים. לא redesign ידני של כל asset. קודם normalization/dedup; inventory + identity + provenance + mapping report. אסור להתחיל ליצור מחדש Knowledge Assets שכבר קיימים לפני שיוצאים מה יס.

P1 — להשלים Runtime/Release foundation. tiny launcher, exact pinning, digest verification, restore/doctor/rollback. כבר כאן להבטיח שה־artifact שנבדק הוא זה שמופעל.

P1 — להכניס את corpus הישן למודל החדש. זה קורה לפני פיתוח Knowledge Base חדש. ה־goal הוא "use what exists" ולא "rebuild our library".

P1 — Project Profile + Resolver CLI. להגיע כמה שיותר מהר לנקודה שבה corpus קיים נותן תשובה שימושית ל־task אמיתי, לפני UI או agent automation עשיר.

P1 — Telemetry מוקדם. לפני vertical slice עם agent, כדי שה־development של EOS עצמו יתחיל לייצר evidence ולא להגיע ל־observability רק בסוף.

P2 — vertical slice אמיתי. agent → resolve → inspect → work → tests → telemetry → evidence → investigation. זה checkpoint הארכיטקטוני החשוב ביותר. אם הוא אינו טבעי, עוצרים ומפשטים לפני D9–D15 הרחבים.

P2 — Assurance + canonical learning. רק לאחר שיש outcomes שאפשר למדוד באמת.

P3 — External ecosystem + multi-agent. capability-driven integration, Claude/Codex parity, live verification, MCP/native/plugin/CLI discovery

P3 — resilience, adversarial and scale. fault injection, poisoning, credential boundary, corrupted release, backend outage, data replay

P4 — native-vs-assisted לפני EOS real rollout. חייב להראות incremental value או לפחות לא לפגוע, ורכיבים מיותרים הופכים removal candidates

P4 — Project 8 canary project שני RC → Stable

הסיכונים המיידיים ביותר לפני תחילת הקוד

קריטית

סיכון: בניית מערכת גדולה לפני vertical proof

מניעה: Stage 8 מוקדם; no UI/platform expansion לפניו

קריטית

סיכון: privileged Supabase credential בסביבת agent

מניעה: scoped ingestion/RLS/backend boundary; no secret key to agent

קריטית

סיכון: importing old EOS architecture עם assets

מניעה: clean implementation; old code reference-only

גבוהה

סיכון: duplicate knowledge חוזר תחת schemas חדשים

מניעה: Solution Sets + dedup + one Champion

גבוהה

סיכון: evals שבורים נותנים false confidence

מניעה: evaluator controls, holdouts, task validity review

גבוהה

סיכון: learning loop מאמן ובוחן על אותה evidence

מניעה: origin classes + independence groups

גבוהה

סיכון: MCP/plugin/provider detail דולף ל-core

מניעה: capability abstraction + adapters

גבוהה

סיכון: stale pinned data נהיה

מניעה: live overlay + future promotion

גבוהה

סיכון: Release tag אינו באמת artifact שנבדק

מניעה: digest + immutable release + provenance verification

בינונית

סיכון: telemetry schema ננעל על Claude/Codex 2026

מניעה: EOS envelope + generic causal fields + adapter mapping

בינונית

סיכון: observability נהיה cost/context burden

מניעה: explicit overhead budgets

בינונית

סיכון: Supabase storage backup assumption שגויה

מניעה: separate Storage restore plan if Storage adopted

בינונית

סיכון: champion הופך monopoly היסטורי

מניעה: challengers + periodic shadow/challenge evaluation

פרמטרים שבכוונה נשארים להכרעה מבוססת נתונים

Raw diagnostic telemetry retention ו-cold-storage policy.

RPO/RTO של Evidence Plane.

Asset Score prior, weights ו-decay.

Minimum independent evidence לקידום Champion.

Margin/confidence הנדרש להחלפת Champion.

Freshness TTLs לכל class/provider/fact type.

Candidate promotions יכולים בעתיד להתמזג אוטומטית.

Performance budgets ל-startup/resolve/SQLite/Supabase.

Token/context/tool-call overhead budget.

stopping rules לפי eval class ומספר trials ו-.

Release bake duration/ring promotion thresholds.

reproducibility-critical Evidence של גודל retention.

Scale thresholds לאחר שנמדד usage אמיתי.

Owner Defaults מוצדק מעבר ל-project-specific config.

השארת הפרמטרים האלה פתוחים אינה חור בתכנון. ההפך: קיבוע מספרים ללא baseline יהפוך assumptions שרירותיים לחוב טכני. הם צריכים להיות **versioned policies** עם **provenance** של הסיבה לשינוי.

Source-of-truth note למסמך עצמו: העובדות החיצוניות הרגישות לזמן בדוח — לדוגמה מצב MCP Registry כ־2026-07-28, preview, MCP specification 2026-07-28, תוכנית deprecation הנוכחית של OpenAI Evals, והמעבר של Supabase מ־legacy API keys — נכונות לפי התיעוד הרשמי שנבדק ב־4 בספטמבר 2026, ולא צריכות להפוך ל־hard-coded assumptions של implementation.

הגדרת ההצלחה הסופית של Improved-Engineering-OS אינה "כל 7 1 Decisions מומשו בקוד".
ההגדרה היא:

Agents work naturally
+
useful existing knowledge is reused
+
the right knowledge is surfaced with low noise
+
critical risks are not silently missed
+
real outcomes create trustworthy evidence
+
evidence improves future recommendations
+
one best proven solution is visible per equivalent problem
+
state and releases are reproducible/recoverable
+
new tools/providers/models do not require core redesign
+
EOS overhead remains justified
+
obsolete EOS layers can be removed as AI improves

במילים אחרות, Improved-Engineering-OS צריך להיות **מערכת של זיכרון הנדסי, ראיות, התאמה ובטיחות שמתחזקת עם הזמן — לא שכבת שליטה שדורשת מה־AI של העתיד להמשיך לעבוד כמו ה־AI של**
2026.

הערת מקורות

המסמך מבוסס על הדוח שסופק בשיחה. סימוני הציטוט הפנימיים של סביבת ChatGPT הוסרו מן הגרסה העצמאית כדי למנוע קישורים שבורים. בעת הכנסת המסמך ל־repository מומלץ לצרף רשימת מקורות/קישורים רשמית בהתאם ל־sources_verified_at ול־provenance של גרסת הארכיטקטורה.